

Advanced JS

Scope

① Block Scope

Variables declared inside a pair of curly braces cannot be accessed from outside the block

② Function Scope

Variables declared inside a function cannot be accessed from outside the function

③ Global Scope

Globally scoped variables can be accessed inside a block or function

Closure

- A closure is the combination of a function bundled together with references to its surrounding state.
- Closures are created every time a funcⁿ is created, at creation time
- In Javascript, when we return a function from another function, we are effectively returning a combination of the funcⁿ definition along with the function's scope. This would let the function definition have an associated persistent memory which could hold on to live data between executions. That combination of the function and its scope chain is what is called a closure in Javascript

Function Currying

→ Currying is a process in functional programming in which we transform a function with multiple arguments into a sequence of nesting functions that take one argument at a time

→ function $f(a, b, c)$ is transformed to $f(a)(b)(c)$

this keyword

→ The JavaScript this keyword which is used in a function, refers to the object it belongs to

→ It makes the functions reusable by letting us decide the object value

→ this value is determined entirely by how a function is called

How to determine this

- ① Implicit binding
- ② Explicit binding
- ③ New binding
- ④ Default binding

Order of Precedence

New binding
Explicit binding
Implicit binding
Default binding



Iterables & Iterators

Iterables

String Code

```
const str = 'Vishwas'  
for (let i = 0; i < str.length; i++) {  
  console.log(str.charAt(i))  
}
```

Array Code

```
const arr = ['V', 'i', 's', 'h', 'w', 'a', 's']  
for (let i = 0; i < arr.length; i++) {  
  console.log(arr[i])  
}
```

Problems in ~~At~~ this

- ① Difficulty in accessing the element
- ② Difficult to manage iteration on the data for various types of data structures

There was a need to iterate over ~~this~~ various data structures in a new way that abstracts away the complexity of accessing elements one by one & was at the same time uniform across the different data structures

Solution

For of Loop

String Code

```
const str = 'Vishwas'  
for (const char of str) {  
  console.log(char)  
}
```

Array Code

```
const arr = ['V', 'i', 's', 'h', 'w', 'a', 's']  
for (const item of arr) {  
  console.log(item)  
}
```


- An object which implements the iterable protocol is called an iterable
- For an object to be an iterable it must implement a method at the key [symbol.iterator]
- The method should not accept any argument & should return an object which conforms to the iterator ~~protocol~~ protocol
- The iterator protocol decides whether an object is an iterator)
- The object must have a next() method that returns an object with 2 properties
 - value: which give the current element
 - done: which is a boolean value indicating whether or not there are any more elements that could be iterated upon.

A Synchronous Javascript

→ In its most basic form, Javascript is a synchronous, blocking, single-threaded language

Synchronous

→ If we have 2 functions which log messages to the console, code executes top to down, with only one line executing at any given time

Blocking

→ No matter how long a previous process takes, the subsequent processes won't kick off until the former is completed

→ If function A had to execute an intensive chunk of code, Javascript has to finish that without moving on to function B. Even if that code takes 10 seconds or 1 minute

Single-threaded

→ A ~~single~~ thread is simply a process that our javascript program can use to run a task

→ Each thread can only do one task at a time

→ Javascript has just the one thread called the main thread for executing any code

- So now asynchronous Javascript is developed & is used.
- Now, we can let our code do several things at the same time without stopping or blocking our main thread

setTimeout()

- The `setTimeout()` function executes a particular block of code once after a specified time has elapsed

→ Syntax

`setTimeout(function, duration, param1, param2, ...)`

- The first parameter is a function to run, or a reference to a function defined elsewhere
- The second parameter is a number representing the duration in milliseconds to wait before executing the code.
- After the second parameter, we can pass in zero or more values that represent any parameters we want to pass to the funcⁿ when it runs

```
→ function greet() {  
  console.log('Hello');
```

```
}
```

`setTimeout(greet, 2000);` → Logs Hello to console after 2 seconds

```
→ function greet(name) {  
  console.log('Hello ${name}');
```

```
}
```

`setTimeout(greet, 2000, 'Vishwas');` → Logs Hello Vishwas to the console after 2 seconds

→ To clear a timeout, we can use the `clearTimeout()` method passing in the identifier returned by `setTimeout` as a parameter

→ Eg

```
function greet() {  
  console.log("Hello");  
}  
  
const timeoutId = setTimeout(greet, 2000)  
clearTimeout(timeoutId);
```

→ Nothing is logged to the console

→ A more practical scenario is clearing timeouts when the component is unmounting to free up the resources & also prevent code from incorrectly executing on an unmounted component.

setInterval()

→ The `setInterval()` function repeatedly runs the same code over & over again at regular intervals

→ Syntax

`setInterval(function, duration, param1, param2, ...)`

- The 1st parameter is the code to execute
- The 2nd parameter is the duration in milliseconds
- After the 2nd parameter, we can pass in zero or more values that represent any parameters we want to pass to the function when it is run

→ `function greet() {
 console.log('Hello');
}
setInterval(greet, 2000);`
Logs Hello every 2 seconds

→ Intervals keep running a task forever so we should clear the interval when appropriate.

`eg const intervalId = setInterval(greet, 2000, 'Vishwas');
clearInterval(intervalId);`

Imp Points

- ① Timers & intervals are not a part of JavaScript itself. They are implemented by the browser & `setTimeout` and `setInterval` are basically names given to that functionality in JavaScript.
- ② Duration parameter is the minimum delay, not guaranteed delay

- ③ It is possible to achieve the same effect as `setInterval` with a recursive `setTimeout`.

Recursive setTimeout

```
setTimeout(function run() {  
  console.log('Hello');  
  setTimeout(run, 100);  
, 100)
```

setInterval

```
setInterval(function run() {  
  console.log('Hello');  
, 100)
```


Difference betⁿ recursive & setTimeout & setInterval

Recursive setTimeout

- ① Duration is guaranteed betⁿ executions irrespective of how long the code takes to run, the interval will remain the same

- ② we can calculate a different delay before running each iteration

setInterval

- ① The duration interval includes the time taken to execute the code we want to run

The code takes 40 ms to run, the interval is 60 ms

The code takes 50 ms to run, the interval is 50 ms

- ② setInterval is always a fixed interval duration

Callbacks

- In JavaScript, functions are first class objects
- Just like an object, a function can be passed as an argument to a function
- A function can also be returned as values from other functions.

```
eg function greet(name) {  
  console.log('Hello ' + name);  
}  
  
function higherOrderFunction(callback) {  
  const name = 'Vishwas';  
  callback(name);  
}  
  
higherOrderFunction(greet);
```

→ Any function that is passed as an argument to another function is called a callback function in JavaScript

→ The function which accepts a function as an argument or returns a function is called a higher order function.

Synchronous Callbacks

→ A callback which is executed immediately is called a synchronous callback

```
→ eg function greet(name) {  
    console.log('Hello ${name}');  
}  
  
function higherOrderFunction(callback) {  
    const name = 'Vishwas';  
    callback(name);  
}  
  
higherOrderFunction(greet);
```

```
→ eg let numbers = [1, 2, 3, 4, 5, 3, 6]  
    numbers.sort((a, b) => a - b);  
    numbers.map(n => n * 2);  
    numbers.filter(n => n % 2 == 0);
```


Asynchronous callbacks

- An asynchronous callback is a callback that is often used to continue or resume code execution after an asynchronous operation has completed
- Callbacks are used to delay the execution of a function until a particular time or event has occurred
- Data fetching takes time & we can only run the func we want to after the data has been fetched & not immediately

```
eg function greet(name) {  
    console.log('Hello ' + name);  
}  
setTimeout(greet, 2000, 'Vishwas');
```

```
eg function callback() {  
    document.getElementById("demo").innerHTML = "Hello World";  
}  
document.getElementById("btn").addEventListener("click",  
    callback);
```

```
eg $.get("url", function(data) {  
    $("result").html(data);  
    alert("Load was performed");  
})
```

- callback functions allow us to delay the execution of a function

Problems with callbacks pattern

- If we have multiple callback functions where each level depends on the result obtained from the previous level, the nesting of functions become so deep that the code becomes difficult to read & maintain
- This leads to callback hell
- An alternative & the recommended approach now is to use Promises

Promises

In layman terms - Dinner Scenario

- Consider a scenario where you & your roommate want to have dinner at home
- you want to prepare your special soup.
- At the same time, you feel like having tacos from the food truck nearby
- You ask your roommate, "Hey can you go down to the food truck & get us some tacos?"
- When he is about to leave, you tell him
 - There is no point in me waiting till you're back to prepare the soup. So, I'll start with the soup now but when you reach the place, can you promise that you'll text me so I can start setting up the dining table?"
 - Also let me know if something goes wrong. If you can't find the food truck or if they are out of tacos for the night. Let me know that you can't get the tacos & I'll start cooking some pasta instead.

→ Your friend says "Sure, I promise. I'll head out now and text you in some time."

→ Now, you go about preparing your soup but the status on tacos? We can say that it is currently pending till you receive that message from your friend.

→ When you get back a text message saying that he is getting the tacos, your desire to eat tacos has been fulfilled. You can then proceed to set up the table.

→ If the text message says that he can't bring back any tacos, your desire to have tacos have been rejected & you now have to cook some pasta instead.

Dinner Scenario

Javascript

① Your friend

① Promise

② While your friend is on his way to the food truck, you know that it could take a while & while you don't want to sit idle, so you start preparing soup in the meantime

② Asynchronous operation in javascript (`fetchTacos`)

③ When your friend text's you with "Can get tacos / Can't get tacos", it answers your quesⁿ on whether he is getting the taco or not

③ Promise return value

④ Can get tacos

④ Promise is said to be fulfilled

⑤ cannot get tacos

⑤ Promise is said to be rejected

⑥ Set up the table

⑥ Success callback

⑦ Cook pasta

⑦ Failure callback

- A promise is a proxy for a value not necessarily known when the promise is created. It allows us to associate handlers with an asynchronous action's eventual success value or failure reason.
- A promise is simply an object in JavaScript
- A promise is always in one of the 3 states
 - pending: which is initial state, neither fulfilled nor rejected
 - fulfilled: meaning that the operation completed successfully
 - rejected: meaning that the operation failed
- Promises help us deal with asynchronous code in a far more simpler way compared to callbacks
- callback hell can be avoided with promises

How to work with Promises

Dinner Scenario

- ① your friend
- ② Can get tacos/cannot get tacos
- ③ can get tacos
- ④ Cannot get tacos
- ⑤ Set up the table
- ⑥ Cook pasta

Javascript

- ① Promise
- ② Promise value
- ③ Fulfill promise
- ④ Reject promise
- ⑤ Success callback
- ⑥ Failure callback

How to create a Promise

```
const promise = new Promise()
```

How to fulfill or reject the Promise

```
const promise = new Promise((resolve, reject) => {
```

```
})
```

Both are fun^{ct}

```
const promise = new Promise((resolve, reject) => {  
  // change status from 'pending' to  
  'fulfilled'
```

```
  resolve()
```

```
})
```

```
const promise = new Promise((resolve, reject) => {  
  // change status from 'pending' to 'rejected'
```

```
  reject()
```

```
})
```

```
const promise = new Promise((resolve, reject) => {
```

```
  setTimeout(() => {
```

```
    // Food truck found
```

```
    // change status from 'pending' to 'fulfilled'
```

```
    resolve()
```

```
  }, 5000)
```

```
})
```

```
const promise = new Promise((resolve, reject) => {
```

```
  setTimeout(() => {
```

```
    // Food truck not found
```

```
    // change status from 'pending' to 'rejected'
```

```
    reject()
```

```
  }, 5000)
```

```
})
```

- Promise status : pending to fulfilled => then() is executed
- Promise status : pending to rejected => catch() is executed

How to execute callback functions based on the status change?

```
const onFulfillment = (result) => {  
  console.log(result);  
  console.log('Set up the table to get tacos');
```

```
}  
onRejection = (error) => {  
  console.log(error);  
  console.log('start cooking pasta');
```

```
}  
// Resolve scenario  
const promise = new Promise((resolve, reject) => {  
  setTimeout(() => {
```

// Food truck found

// Change status from 'pending' to 'fulfilled'

resolve('Bringing tacos');

}, 5000)

```
}  
// Reject scenario  
const promise = new Promise((resolve, reject) => {
```

setTimeout(() => {

// Food truck not found

// Change status from 'pending' to 'rejected'

reject('Not bringing tacos. Food truck not there.')

}, 5000)

```
}  
promise.then(onFulfillment)  
promise.catch(onRejection)
```

O/P

Bringing tacos
set up the table to eat tacos

cannot bring tacos
start cooking pasta

then() funcⁿ

- then() funcⁿ can accept both success & error callbacks but ~~it~~ is not preferred over using catch funcⁿ

Chaining Promises

- Both then & catch method return promises
- then() & catch() methods can be chained in JavaScript
- `promise.then(onFulfillment).catch(onRejection)`

Promise - Static methods

① Promise.all()

- query multiple APIs & perform some actions but only after all the APIs have finished loading

→ `const promise1 = Promise.resolve(3);`

`const promise2 = 42;`

```
const promise3 = new Promise((resolve, reject) => {  
  setTimeout(resolve, 100, 'foo');  
});
```

```
Promise.all([promise1, promise2, promise3]).then((values) => {  
  console.log(values);  
});
```

// expected output: A array [3, 42, "foo"]

- The Promise.all() method takes an iterable of promises as an input & returns a single Promise that resolves to an array of the results of the input promises.

→ Returned promise will resolve when all of the input's promises have resolved, or if the input iterable contains no promises

→ It rejects immediately if any of the input promises reject or the non promises throw an error, & will reject with this first rejection message/error

② Promise.allSettled()

→ Promise.allSettled() waits for all input promises to complete regardless of ~~what~~ whether or not one of them is rejected

→ Eg

```
const promise1 = Promise.resolve(3);  
const promise2 = 42;  
const promise3 = new Promise((resolve, reject) => {  
  setTimeout(resolve, 100, 'foo');  
});  
Promise.allSettled([promise1, promise2, promise3]).then((values) => {  
  console.log(values);  
});  
// expected output: Array[3, 42, "foo"]
```

③ Promise.race()

→ The `Promise.race()` method returns a promise that fulfills or rejects as soon as one of the input promises fulfills or rejects, with the value or reason from that promise

```
→ eg const promise 1 = new Promise((resolve, reject) => {  
    setTimeout(resolve, 500, 'one');  
});
```

```
const promise 2 = new Promise((resolve, reject) => {  
    setTimeout(resolve, 100, 'two');  
});
```

```
Promise.race([promise 1, promise 2]).then((value) => {  
    console.log(value);  
}); // Both resolve, but promise 2 is faster
```

// expected output: "two"

Async

→ The `async` keyword is used to declare async functions

→ Async functions are functions that are instances of the `AsyncFunction` constructor

→ Unlike normal functions, async functions always return a promise

Normal func

```
function greet() {  
    return "Hello"  
}  
  
greet()  
// O/P  
"Hello"
```

async func

```
async function greet() {  
    return "Hello"  
}  
  
greet()  
// O/P  
Promise {<fulfilled>: "Hello"}
```


await

- await keyword can be put in front of any async promise based function to pause your code until that promise settles & returns its result
- await only works inside async functions, cannot use await in normal functions

```
→ eg async function greet() {  
  let promise = new Promise((resolve, reject) => {  
    setTimeout(() => resolve("Hello"), 1000)  
  });  
  let result = await promise; // wait until the promise resolves  
  console.log(result); // "Hello"  
}
```

```
→ eg async function fetchData() {  
  try {  
    const user = await fetchCurrentUser('api/user')  
    const followers = await fetchFollowersByUserId('api/followers/${result.userId}')  
    const interests = await fetchFollowersInterests('api/interests/${result.followerId}')  
    const tags = await fetchInterestTags('api/tags/${result.interestId}')  
    const description = await fetchTagDescription('api/description/${result.tagId}')  
    console.log('Display the data', result)  
  }  
  catch(e) {  
    console.log('Error', e);  
  }  
}
```

Sequential execution

```
function resolveHello() {  
  return new Promise (resolve => {  
    setTimeout (function () {  
      resolve ('Hello')  
    }, 2000)  
  })  
}
```

```
function resolveWorld() {  
  return new Promise (resolve => {  
    setTimeout (function () {  
      resolve ('World')  
    }, 1000)  
  })  
}
```

>
// Sequential execution

```
async function sequentialStart() {  
  const hello = await resolveHello()  
  console.log(hello); // Logs after 2 seconds  
  const world = await resolveWorld()  
  console.log(world) // Logs after 2 + 1 = 3 seconds  
}
```

>
sequentialStart()

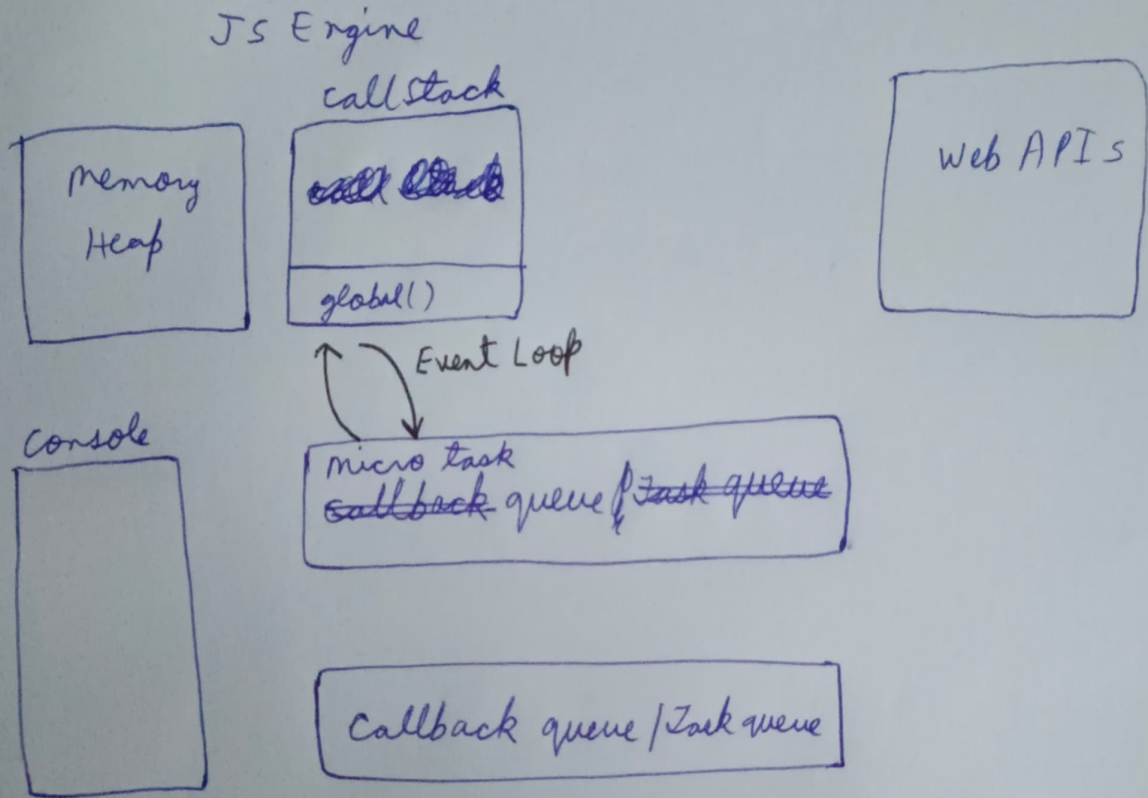
// concurrent execution

```
async function concurrentStart() {  
  const hello = resolveHello()  
  const world = resolveWorld()  
  console.log(await hello); // Logs after 2 seconds  
  console.log(await world); // Logs after 2 seconds  
}
```

>
concurrentStart()

Event loop

JS runtime environment



Micro task queue has more priority than callback queue